



Surviving the MongoDB Viva

The definitive interview dossier on Projection, Sort, Skip, and Limit.

FROM BASIC SYNTAX TO
BULLETPROOF EXECUTION ORDER

MODIFIER	PLAIN ENGLISH	SYNTAX SNAPSHOT	REAL-WORLD USE CASE
 Projection	Includes or excludes specific fields.	<code>{name: 1, age: 0}</code>	Hiding passwords or sensitive data from API payloads.
 Sort	Orders the returned results.	<code>sort({age: -1})</code>	Showing the most recently created user first.
 Skip	Bypasses a set number of documents.	<code>skip(10)</code>	Moving to Page 2 of a data grid.
 Limit	Restricts the total count of returned documents.	<code>limit(5)</code>	Fetching only a fixed batch of results to save bandwidth.

Projection: The Binary Rule

Include fields:

```
db.students.find({}, {name: 1, age: 1})
```

Exclude fields:

```
db.students.find({}, {age: 0})
```

Mix and Match:

```
db.students.find({}, {name: 1, age: 0})
```



The `_id` Exception

The `_id` field is unique and included by default. Limit does not work to hide it.

If you do not want it in your result set, you **MUST** explicitly exclude it.

```
db.students.find({}, {age: 1,  
_id: 0})
```

Sorting Logic: Directional Flags



`sort({age: 1})`
Smallest to largest / A to Z.



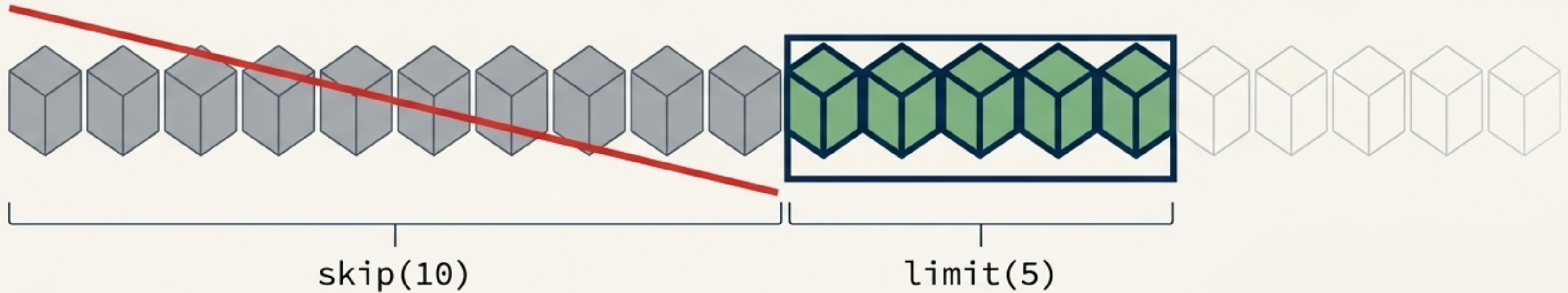
`sort({age: -1})`
Largest to smallest / Z to A.

Sorting by multiple criteria

```
db.students.find({}).sort({name: 1, age: -1})
```

Sorts by name ascending first. If names match, it breaks the tie by sorting age descending.

Slicing the Result Set



```
db.students.find().skip(10).limit(5)
```

MongoDB bypasses the first 10 documents entirely, then captures exactly the next 5. The rest are ignored.

The Pagination Playbook

The primary real-world application of Skip and Limit.

ID	Name	Date	Status
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

< Page 1 | Page 2 | **[Page 3]** >

Equation Breakdown

Page size = 5

Target Page = 3

Items to skip = (Target Page - 1) * Page size

Items to skip = (3 - 1) * 5 = 10



```
db.students.find().skip(10).limit(5)
```

**Skip jumps over previous pages;
Limit enforces the page size.**

The Syntax Illusion

```
db.students.find({})  
  .skip(2)  
  .limit(2)
```

=

```
db.students.find({})  
  .limit(2)  
  .skip(2)
```

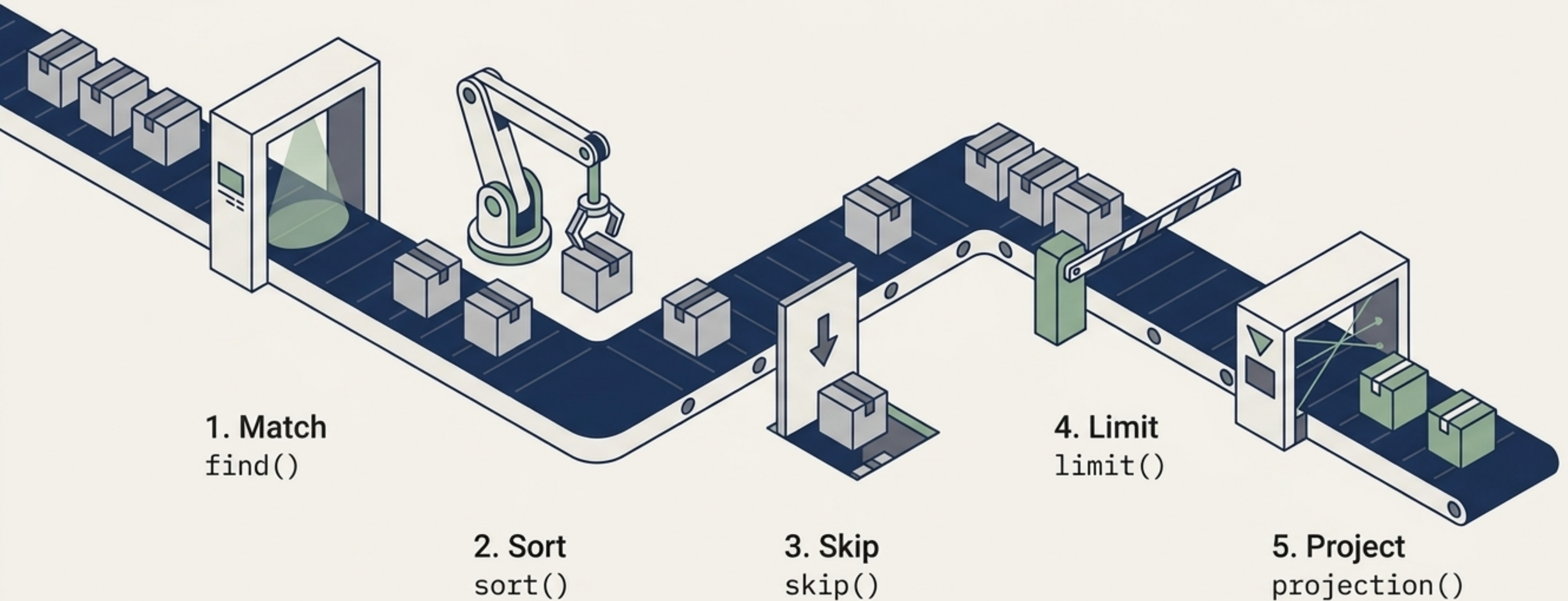


In standard JavaScript, the order of chained methods dictates execution.
In MongoDB, it does not matter.

**Limit order at the end or skip order at the first makes zero difference.
The database engine enforces its own strict internal sequence.**

The Unbreakable Execution Order

Regardless of **chaining order** in Node.js, the engine processes data through this exact pipeline.





Make } (Match / find)
Sure }
Students } (Sort)
 } (Skip)
Learn } (Limit)
Projection } (Project)

Burn this acronym into your memory. When the examiner asks about execution order, recite this exact sequence.

THE GRILLING: Q1

Candidate, if I chain **.limit(2).skip(5)** in my Node.js backend, what exactly happens in the database?

BULLETPROOF RESPONSE

MongoDB **ignores the chained order**. It will always execute the **Sort** first, then **Skip**, then **Limit**. It will confidently **skip** the first 5 documents, and then **limit** the final output to **2**.

THE GRILLING: Q2

I wrote a query to only project the `name` field using `{name: 1}`. But the `_id` keeps showing up in my JSON payload. Why, and how do I fix it?

BULLETPROOF RESPONSE

The `_id` field is uniquely exempt from standard projection rules; it is included by default. To remove it, you must explicitly exclude it by adding `{ _id: 0 }` to your projection object.

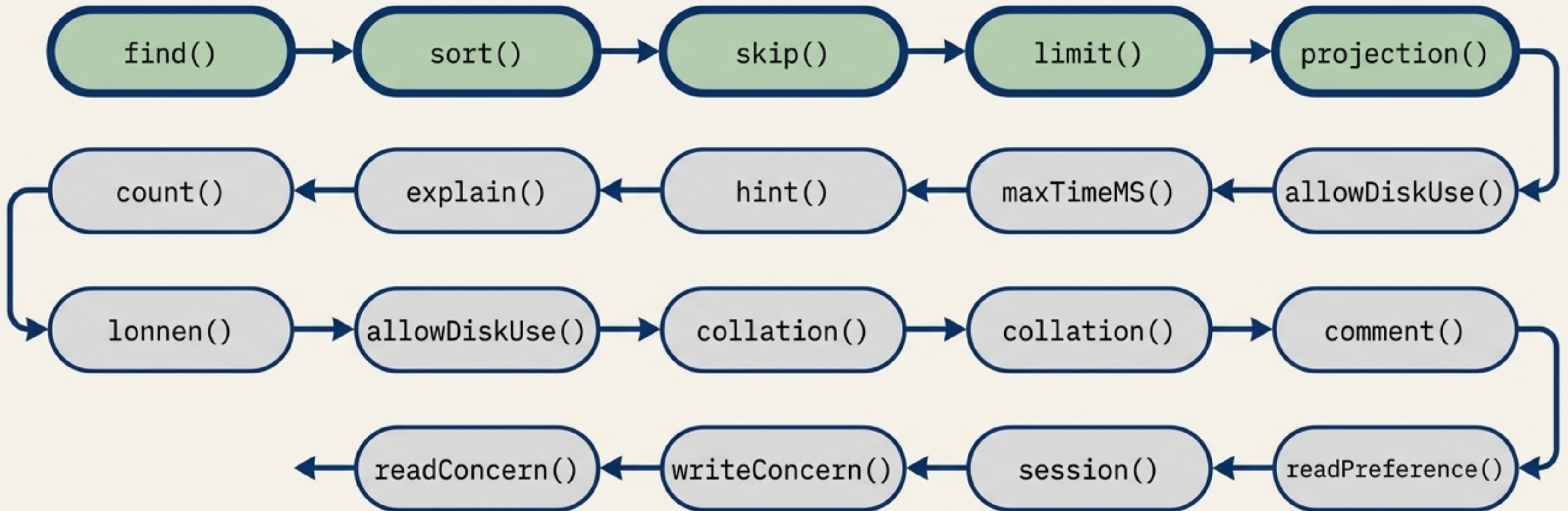
THE GRILLING: Q3

What is the purpose of the empty braces `{}` in `db.students.find({})`, and what happens if I accidentally pass that same empty object to `deleteMany({})`?

BULLETPROOF RESPONSE

The `{}` is the default empty filter document. In `find()`, writing it is optional but good practice to explicitly show no filters are applied. In `deleteMany({})`, it acts as a wildcard filter and will permanently delete all documents in the collection.

The Complete Sequence (Beyond the Basics)



Further Study
Official MongoDB Manual
MongoDB Node.js Driver Documentation

**Preparation eliminates panic.
Good luck.**